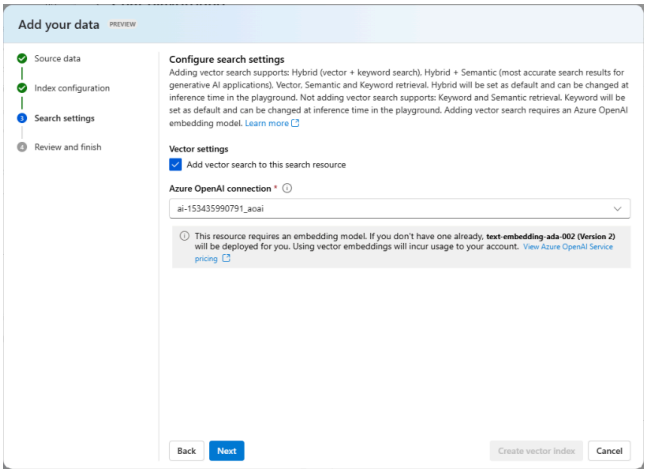
Develop a RAG-based solution with your own data using Microsoft Foundry

Thursday, March 12, 2026 11:19 AM

1. RAG is a technique that you can use to ground a language model. In other words, it's a process for retrieving information that is relevant to the user's initial prompt. In general terms, the RAG pattern incorporates the following steps:
   a. Retrieve grounding data based on the initial user-entered prompt.
   b. Augment the prompt with grounding data.
   c. Use a language model to generate a grounded response.
2. By retrieving context from a specified data source, you ensure that the language model uses relevant information when responding, instead of relying on its training data.
3. You can use Microsoft Foundry to build a custom agent that uses your own data to ground prompts. Microsoft Foundry supports a range of data connections that you can use to add data to a project, including:
   a. Azure Blob Storage
   b. Azure Data Lake Storage Gen2
   c. Microsoft OneLake
4. When you build an agent with the Microsoft Foundry, you can use the integration with Azure AI Search to retrieve the relevant context in your chat flow.
5. Azure AI Search is a retriever that you can include when building a language model application with prompt flow. Azure AI Search allows you to bring your own data, index your data, and query the index to retrieve any information you need.
6. While a text-based index will improve search efficiency, you can usually achieve a better data retrieval solution by using a vector-based index that contains embeddings that represent the text tokens in your data source. An embedding is a special format of data representation that a search engine can use to easily find the relevant information. More specifically, an embedding is a vector of floating-point numbers.
7. The distance between vectors can be calculated by measuring the cosine of the angle between two vectors, also known as the cosine similarity. In other words, the cosine similarity computes the semantic similarity between documents and a query.
8. When you want to be able to use vector search to search your data, you need to create embeddings when creating your search index. To create embeddings for your search index, you can use an Azure OpenAI embedding model available in Microsoft Foundry.
9. In Azure AI Search, a search index describes how your content is organized to make it searchable.
10.
11. 

12. How you configure your search index depends on the data you have and the context you want your language model to use. For example, keyword search enables you to retrieve information that exactly matches the search query. Semantic search already takes it one step further by retrieving information that matches the meaning of the query instead of the exact keyword, using semantic models. Currently, the most advanced technique is vector search, which creates embeddings to represent your data. It uses mathematical representations of text (vectors) to find similar documents or passages based on their semantic meaning or context.
13. Hybrid search: Combines any or all of the other search techniques. Queries are executed in parallel and are returned in a unified result set.
14. Hybrid search is a combination of keyword (and full text), and vector search, to which semantic ranking is optionally added. When you create an index that is compatible with hybrid search, the retrieved information is precise when exact matches are available (using keywords), and still relevant when only conceptually similar information can be found (using vector search).
15. **Create a RAG-based client application**
    a. When you've created an Azure AI Search index for your contextual data, you can use it with an OpenAI model. To ground prompts with data from your index, the Azure OpenAI SDK supports extending the request with connection details for the index.
    b.

```python
from openai import AzureOpenAI

# Get an Azure OpenAI chat client
chat_client = AzureOpenAI(
    api_version = "2024-12-01-preview",
    azure_endpoint = open_ai_endpoint,
    api_key = open_ai_key
)

# Initialize prompt with system message
prompt = [
    {"role": "system", "content": "You are a helpful AI assistant."}
]
```

c.
```python
# Add a user input message to the prompt
input_text = input("Enter a question: ")
prompt.append({"role": "user", "content": input_text})

# Additional parameters to apply RAG pattern using the AI Search index
rag_params = {
    "data_sources": [
        {
            "type": "azure_search",
            "parameters": {
                "endpoint": search_url,
                "index_name": "index_name",
                "authentication": {
                    "type": "api_key",
                    "key": search_key,
                }
            }
        }
    ],
}

# Submit the prompt with the index information
response = chat_client.chat.completions.create(
    model="<model_deployment_name>",
    messages=prompt,
    extra_body=rag_params
)

# Print the contextualized response
completion = response.choices[0].message.content
print(completion)
```

d. In this example, the search against the index is keyword-based - in other words, the query consists of the text in the user prompt, which is matched to text in the indexed documents.

e. To use a vector-based query, you can modify the specification of the Azure AI Search data source details to include an embedding model; which is then used to vectorize the query text.

f.

g.
```python
rag_params = {
    "data_sources": [
        {
            "type": "azure_search",
            "parameters": {
                "endpoint": search_url,
                "index_name": "index_name",
                "authentication": {
                    "type": "api_key",
                    "key": search_key,
                },
                # Params for vector-based query
                "query_type": "vector",
                "embedding_dependency": {
                    "type": "deployment_name",
                    "deployment_name": "<embedding_model_deployment_name>",
                },
            }
        }
    ],
}
```

16. Implement RAG in a prompt flow:

a. After uploading data to Microsoft Foundry and creating an index on your data using the integration with Azure AI Search, you can implement the RAG pattern with Prompt Flow to build a generative AI application.

b. The flow is then defined as a series of connected tools, each of which performs a specific operation on the inputs and other environmental variables. There are multiple types of tool that you can include in a prompt flow to perform tasks such as:

   i. Running custom Python code

   ii. Looking up data values in an index

   iii. Creating prompt variants - enabling you to define multiple versions of a prompt for a large language model (LLM), varying system messages or prompt wording, and compare and evaluate the results from each variant.

   iv. Submitting a prompt to an LLM to generate results.

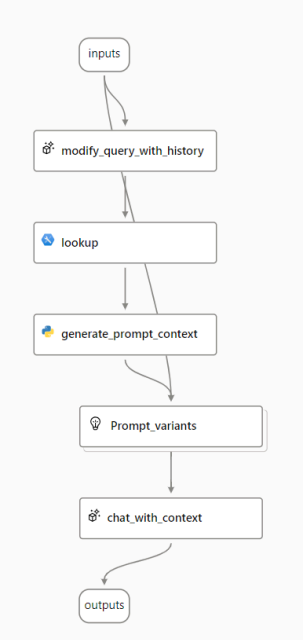
c. Using the RAG pattern in a prompt flow

   i. The key to using the RAG pattern in a prompt flow is to use an Index Lookup tool to retrieve data from an index so that subsequent tools in the flow can use the results to augment the prompt used to generate output from an LLM.

   ii. Prompt flow provides various samples you can use as a starting point to create an application. When you want to combine RAG and a language model in your application, you can clone the Multi-round Q&A on your data sample.

   iii.

   iv.



v. Append the history to the chat input to define a prompt in the form of a contextualized form of a question.

vi. Look up relevant information from your data using your search index.

vii. Generate the prompt context by using the retrieved data from the index to augment the question.

viii. Create prompt variants by adding a system message and structuring the chat history.

ix. Submit the prompt to a language model that generates a natural language response.

x. Step 1: Modify query with history

1. The first step in the flow is a Large Language Model (LLM) node that takes the chat history and the user's last question and generates a new question that includes all necessary information. By doing so, you generate more succinct input that is processed by the rest of the flow.

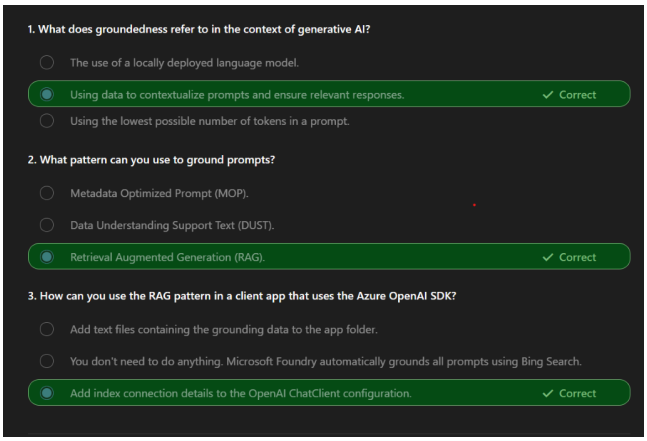
xi. The output of the Index Lookup tool can include the top n results (depending on the parameters you set). When you generate the prompt context, you can use a Python node to iterate over the retrieved documents from your data source and combine their contents and sources into one document string. The string will be used in the prompt you send to the language model in the next step of the flow.

xii. After configuring the sample chat flow to use your indexed data and the language model of your choosing, you can deploy the flow and integrate it with an application to offer users an agentic experience.

17. Assessment:

a.

b.



c.